

Database Management Systems: Transaction Management and Concurrency Control

1. Fundamentals of Transactions

Definition and ACID Concept

A **transaction** is defined as a sequence of database operations performed as a single logical unit of work ^[6]. Every transaction must satisfy the ACID (Atomicity, Consistency, Isolation, Durability) properties to maintain database integrity [Source 2, Source 6]. Among these properties, **durability** ensures that committed changes are permanently saved and supports system recovery after crashes [Source 1, Source 7].

Example: Logical Unit of Work

A classic example of a transaction is a money transfer between two accounts ^[6]:

1. Deduct ₹1000 from Account A ^[6].
2. Add ₹1000 to Account B ^[6].

Both operations executed together form a single logical unit of work ^[6].

Transaction States and Lifecycle Transitions

During its execution, a transaction transitions through several distinct states ^[7]:

- **Active:** The initial state where the transaction is executing its operations ^[7].
- **Partially Committed:** Reached immediately after the last statement of the transaction has been executed ^[7].
- **Committed:** Reached after successful completion; changes are permanently saved into the database ^[7].
- **Failed:** Reached if an error or exception occurs during execution ^[7].
- **Aborted:** Reached after the transaction has been rolled back and the database state restored prior to the transaction's start ^[7].

Transition Rules:

- An **Active** transaction transitions to **Partially Committed** upon executing its final statement, or to **Failed** if an error occurs ^[7].
 - A **Partially Committed** transaction transitions to **Committed** upon saving changes permanently, or to **Failed** if an error occurs ^[7].
 - A **Failed** transaction transitions directly to the **Aborted** state ^[7].
-

2. Schedules and Serializability

Serial vs. Serializable Schedules

A **serial schedule** executes transactions sequentially without interleaving operations [Source 6, Source 7].

A **serializable schedule** is a concurrent schedule whose execution result is equivalent to that of a serial schedule (e.g., executing $T_1 \rightarrow T_2$) [Source 5, Source 6, Source 7].

Conflict Serializability vs. View Serializability

The relationship between conflict serializable schedules and view serializable schedules is defined by a strict inclusion subset:

$$\text{Conflict Serializable} \subset \text{View Serializable}$$

"All Conflict Serializable Schedules are View Serializable, but the Reverse is not True" ^[5].

Reason: Conflict serializable schedules satisfy all conditions required for view serializability, but view serializability includes additional schedules that conflict serializability does not cover ^[5].

Conflict Dependency Analysis

Conflict serializability can be analyzed by identifying conflicting operations across transactions on shared data items ^[8]:

- **Operations on Data Item X :**
 - Same-transaction sequences (e.g., $T_2: \text{read}(X) \rightarrow T_2: \text{write}(X)$) are skipped during dependency evaluation ^[8].
 - If $T_1: \text{read}(X)$ occurs before $T_2: \text{write}(X)$, it creates a conflict precedence order: $T_1 \rightarrow T_2$ ^[8].
- **Operations on Data Item Y :**

- $T_1: \text{write}(Y)$ before $T_2: \text{read}(Y) \implies T_1 \rightarrow T_2$ [8].
 - $T_1: \text{write}(Y)$ before $T_3: \text{read}(Y) \implies T_1 \rightarrow T_3$ [8].
 - $T_1: \text{write}(Y)$ before $T_3: \text{write}(Y) \implies T_1 \rightarrow T_3$ [8].
 - $T_2: \text{read}(Y)$ before $T_3: \text{write}(Y) \implies T_2 \rightarrow T_3$ [8].
-

3. Concurrency Control and Two-Phase Locking (2PL)

Two-Phase Locking (2PL) Protocol

Two-Phase Locking (2PL) is a concurrency control locking protocol designed to govern how transactions acquire and release locks [4].

It consists of two phases:

1. Growing Phase:

- Transactions acquire locks as needed [4].
- Transactions **cannot** release any lock during this phase [4].

1. Shrinking Phase:

- Transactions release locks, but cannot acquire any new locks.

Lock Types and Compatibility Matrix

Concurrency control uses two main types of locks: **Shared** locks and **Exclusive** locks [5]. The interaction between lock requests and currently held locks is dictated by the Lock Compatibility Matrix [5]:

Lock Requested \ Lock Held	Shared	Exclusive
Shared	Yes (Granted) [5]	No (Denied) [5]
Exclusive	No (Denied) [5]	No (Denied) [5]

Lock Requested	Lock Held: Shared	Lock Held: Exclusive

| **Shared** | **Yes** (Granted) [5] | **No** (Denied) [5] |
 | **Exclusive** | **No** (Denied) [5] | **No** (Denied) [5] |

- Multiple transactions can simultaneously hold **Shared** locks on the same data item [5].
- If an **Exclusive** lock is held or requested, no other locks can be granted on that data item [5].

Serializability and Deadlocks in 2PL

- **Serializability Guarantee:** 2PL ensures serializability (✓) ^[4].
- **Deadlock Vulnerability:** 2PL **does not** prevent deadlocks ($\times$) ^[4]. Both transactions can enter a state where they wait forever for each other to release locks ^[4].
- **Requirement:** Separate deadlock detection or deadlock prevention techniques are required alongside 2PL ^[4].

4. Recoverability and Commit Dependency Ordering

Cascading Schedules vs. Recoverable Schedules

In concurrent transaction execution, uncommitted read dependencies can create cascading rollbacks ^[1]:

- **Cascading Schedule Problem:**
 - Sequence: $T_1: W(X) \rightarrow T_2: R(X) \rightarrow T_2: \text{Commit} \rightarrow T_1: \text{Abort}$ ^[1].
 - Problem: T_2 reads uncommitted data written by T_1 . If T_1 aborts, T_2 must also roll back, but T_2 has already committed ^[1].
- **Recoverable Schedule Solution:**
 - Sequence: $T_1: W(X) \rightarrow T_2: R(X) \rightarrow T_1: \text{Commit} \rightarrow T_2: \text{Commit}$ ^[1].
 - **Rule:** If transaction T_2 reads a data item written by transaction T_1 , then T_1 must commit before T_2 commits ^[1].

Constructing Recoverable Schedules via Commit Ordering

If a schedule lacks explicit commit operations, commit statements must be inserted according to transaction read/write dependency order to guarantee recoverability ^[3]:

1. **Order 1:** T_1 commits first (because T_2 and T_3 read data written by T_1) ^[3].
2. **Order 2:** T_3 commits next (because T_4 reads data written by T_3) ^[3].
3. **Order 3:** T_2 commits (since no subsequent operations depend on T_2 's writes after T_1 has committed) ^[3].